

TÀI LIỆU THIẾT KẾ & TRIỂN KHAI

MARKDOWN OUTPUT

Pipeline: PDF / DOCX / Web → Structured Document → Markdown → Qdrant → RAG

Phiên bản: 1.0

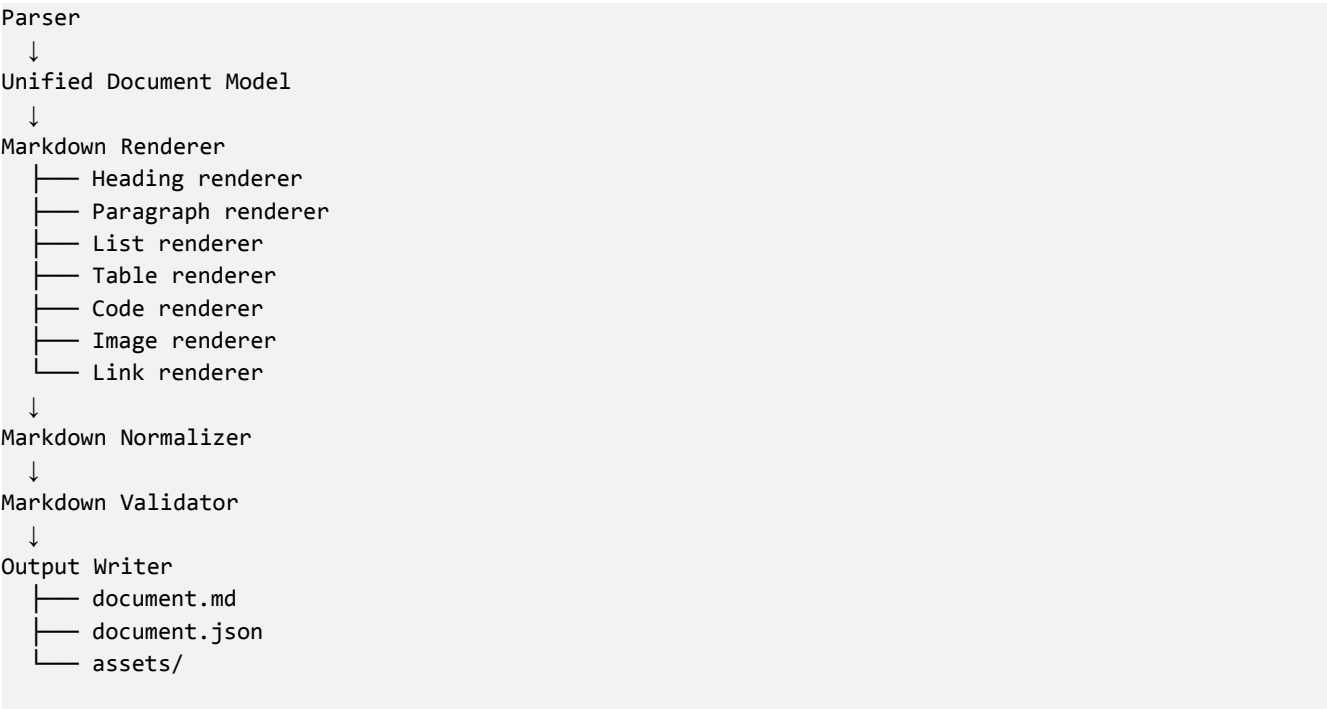
Mục tiêu: Xây dựng module xuất Markdown chuẩn, ổn định và có metadata để làm đầu vào cho pipeline RAG.

1. Phạm vi

Module này chịu trách nhiệm biến dữ liệu tài liệu đã được parser thành các file Markdown có cấu trúc nhất quán. Markdown phục vụ đọc, kiểm tra, lưu trữ và tái sử dụng; dữ liệu cấu trúc và metadata đồng thời được giữ lại để chunking, embedding và truy xuất bằng Qdrant.

- Input chính: kết quả từ Docling/Web Parser hoặc Unified Document Model.
- Output chính: .md.
- Output phụ khuyến nghị: document.json và thư mục assets/.
- Không thực hiện embedding hoặc ghi Qdrant trong module Markdown Renderer.

2. Kiến trúc module



3. Công nghệ

Thành phần	Đề xuất	Vai trò
Parser	Docling	Phân tích PDF/DOCX/HTML và cấu trúc tài liệu

Web extraction	Trafilatura + Playwright	Lấy nội dung chính từ website
Language	Python 3.12+	Triển khai module
Schema	Pydantic	Unified Document Model
Markdown	Custom renderer	Kiểm soát format đầu ra
Validation	Markdown parser/linter	Kiểm tra syntax và cấu trúc
Storage	Local / MinIO / S3	Lưu Markdown và assets

4. Nguyên tắc thiết kế

- Không render Markdown trực tiếp từ raw text nếu parser đã cung cấp document structure.
- Heading phải giữ được hierarchy H1 → H6.
- Table phải được chuẩn hóa về Markdown table; bảng quá phức tạp có thể giữ bản JSON/HTML dự phòng.
- Ảnh phải được lưu thành asset và Markdown tham chiếu bằng relative path.
- Mỗi chunk về sau phải truy ngược được document, section, page và source.
- Không chèn thông tin không có trong nguồn chỉ để làm Markdown đẹp hơn.
- Giữ Unicode UTF-8; đặc biệt lưu ý tiếng Việt, công thức và ký hiệu.
- Renderer phải deterministic: cùng một Document Model phải cho cùng một Markdown.

5. Unified Document Model

```

Document
├── metadata
│   ├── document_id
│   ├── title
│   ├── source_type
│   ├── source_uri
│   ├── language
│   └── created_at
├── blocks[]
│   ├── type: heading | paragraph | list | table | image | code | quote
│   ├── level
│   ├── text
│   ├── page
│   ├── section_path[]
│   └── attributes
└── assets[]

```

6. Chuẩn Markdown đầu ra

6.1 Heading

```

# Tên tài liệu

## Chương 1

### 1.1 Tổng quan

#### 1.1.1 Khái niệm

```

Không dùng nhiều hơn một H1 cho tiêu đề tài liệu nếu nguồn có thể xác định được title.

6.2 Paragraph

Các đoạn văn được ngăn cách bằng một dòng trống. Không để nhiều dòng trống liên tiếp.

6.3 Lists

- Mục 1
- Mục 2
 - Mục con

1. Bước 1
2. Bước 2

6.4 Tables

Cột A	Cột B
---	---
Giá trị 1	Giá trị 2
Giá trị 3	Giá trị 4

Nếu ô chứa xuống dòng, chuẩn hóa thành khoảng trắng hoặc `<br>` tùy policy. Không làm mất nội dung.

6.5 Code

```
```python
def hello():
 return "hello"
```
```

6.6 Images

![Mô tả ảnh](assets/image-001.png)

Tên asset phải ổn định, không chứa ký tự nguy hiểm và không phụ thuộc vào tên file gốc.

6.7 Links

[Tên liên kết](https://example.com)

7. Metadata Front Matter

Khuyến nghị đặt metadata tối thiểu ở đầu Markdown để thuận tiện traceability.

```
---
document_id: doc_001
title: "Tên tài liệu"
source_type: pdf
source: "document.pdf"
language: vi
---
```

Tên tài liệu

Nếu hệ thống RAG lưu metadata trong Qdrant riêng biệt, front matter có thể được bật/tắt bằng cấu hình.

8. Markdown Renderer

```
class MarkdownRenderer:
    def render(self, document) -> str:
```

```

parts = []

for block in document.blocks:
    parts.append(self.render_block(block))

markdown = "\n\n".join(parts)
return MarkdownNormalizer().normalize(markdown)

```

9. Block renderer

```

def render_block(self, block):
    match block.type:
        case "heading":
            return f'{"#" * block.level} {block.text}'
        case "paragraph":
            return block.text.strip()
        case "code":
            lang = block.attributes.get("language", "")
            return f"```{lang}\n{block.text}\n```"
        case "image":
            return f"![{block.text}]({block.attributes["path"]})"
        case "table":
            return render_table(block)
        case _:
            return block.text

```

10. Markdown Normalizer

Normalizer xử lý các lỗi hình thức sau renderer:

- Nhiều blank lines → tối đa một dòng trống.
- Trailing whitespace.
- Heading thiếu khoảng trắng sau #.
- Table separator không hợp lệ.
- Link/image path không hợp lệ.
- Ký tự Unicode không chuẩn hóa nếu policy yêu cầu NFC.
- Code fence không được phá vỡ bởi nội dung bên trong.

```

class MarkdownNormalizer:
    def normalize(self, text: str) -> str:
        text = normalize_unicode(text)
        text = normalize_line_endings(text)
        text = collapse_blank_lines(text)
        text = normalize_headings(text)
        text = normalize_tables(text)
        return text.strip() + "\n"

```

11. Output Writer

```

output/
├── doc_001/
│   ├── document.md
│   └── document.json

```

```
└─ assets/
   └─ image-001.png
   └─ image-002.png
```

```
from pathlib import Path

def write_output(document_id, markdown, metadata, assets):
    root = Path("output") / document_id
    root.mkdir(parents=True, exist_ok=True)

    (root / "document.md").write_text(
        markdown, encoding="utf-8"
    )

    (root / "document.json").write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2),
        encoding="utf-8"
    )
```

12. File naming và directory policy

- document.md: file Markdown chuẩn.
- document.json: metadata + cấu trúc phục vụ traceability.
- assets/: ảnh, biểu đồ hoặc file phụ được Markdown tham chiếu.
- document_id nên là UUID/ULID hoặc ID ổn định từ ingestion service.
- Không ghi đè tài liệu cũ nếu document_id đại diện cho một version; dùng versioning.

13. Traceability cho RAG

Mỗi block/chunk nên có khả năng truy ngược về nguồn.

```
{
  "chunk_id": "chunk_000123",
  "document_id": "doc_001",
  "text": "Nội dung chunk...",
  "source_type": "pdf",
  "source": "security.pdf",
  "page": 42,
  "section_path": [
    "Chương 3",
    "3.2 Firewall"
  ]
}
```

Các trường page, section_path, source và document_id nên được truyền nguyên vẹn sang payload của Qdrant.

14. Chunking sau Markdown

Không nên chunk chỉ theo số ký tự. Ưu tiên semantic/structure-aware chunking.

Markdown

↓

Parse headings
↓
Section tree
↓
Paragraph blocks
↓
Chunk by semantic boundary
↓
Chunk metadata
↓
Embedding
↓
Qdrant

- Không cắt giữa bằng nếu có thể tránh.
- Không cắt code block.
- Ưu tiên giữ heading cùng chunk với nội dung bên dưới.
- Có thể dùng overlap nhỏ giữa các đoạn văn, nhưng không overlap tùy tiện giữa các section.

15. API đề xuất

POST /api/v1/convert

Content-Type: multipart/form-data

Input:

file=<PDF/DOCX>

Response:

```
{
  "document_id": "doc_001",
  "status": "completed",
  "markdown_path": "output/doc_001/document.md",
  "metadata_path": "output/doc_001/document.json"
}
```

POST /api/v1/convert-url

```
{
  "url": "https://example.com/article"
}
```

16. Cấu hình

MARKDOWN_INCLUDE_FRONTMATTER=true
MARKDOWN_INCLUDE_PAGE_MARKERS=true
MARKDOWN_ASSET_DIR=assets
MARKDOWN_MAX_BLANK_LINES=1
MARKDOWN_ENCODING=utf-8
MARKDOWN_STRICT_VALIDATION=true

17. Xử lý lỗi

| Lỗi | Xử lý | Kết quả |
|----------------------------|--------------------|------------------------|
| Parser không đọc được file | Ghi error + source | Không tạo Markdown giả |

| | | |
|-------------------------|---------------------------|-----------------------------|
| Bảng không convert được | Fallback HTML/JSON | Giữ nguyên dữ liệu |
| Ảnh lỗi | Ghi placeholder + warning | Không làm mất toàn document |
| Encoding lỗi | Chuẩn hóa UTF-8 | Output UTF-8 |
| Markdown invalid | Normalize/validate | Fail nếu strict mode |

18. Validation checklist

- File .md tồn tại.
- UTF-8 hợp lệ.
- Title và heading hierarchy hợp lệ.
- Không có blank lines dư thừa.
- Table render đúng.
- Code fence đóng đúng.
- Link/image path hợp lệ.
- Asset tồn tại.
- document_id nhất quán.
- Page/section metadata được bảo toàn.
- Markdown có thể parse lại mà không mất block quan trọng.

19. Unit test đề xuất

```
tests/
├── test_heading_renderer.py
├── test_table_renderer.py
├── test_image_renderer.py
├── test_code_renderer.py
├── test_normalizer.py
├── test_frontmatter.py
├── test_output_writer.py
├── fixtures/
│   ├── simple.pdf
│   ├── complex.pdf
│   ├── scan.pdf
│   └── document.docx
```

20. Acceptance Criteria

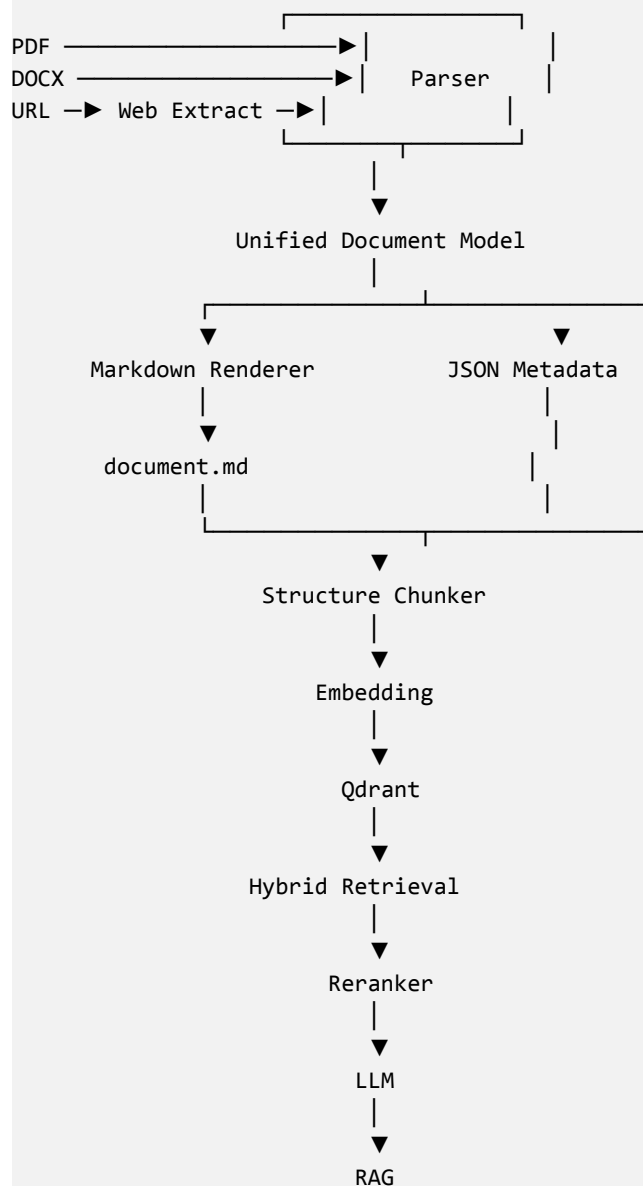
- PDF text-based → Markdown giữ đúng heading, paragraph, table và reading order.
- PDF scan → Markdown có OCR text và metadata page.
- DOCX → Markdown giữ heading, list, table, image và hyperlink.
- Website → Markdown chỉ chứa main content, hạn chế navigation/footer/ads.
- Markdown output deterministic.
- Markdown validator không phát hiện lỗi syntax nghiêm trọng.
- Mọi chunk sau này truy ngược được document_id và source.
- Có thể tái tạo Qdrant payload từ document.json mà không cần đọc lại file nguồn.

21. Roadmap triển khai

1. Phase 1: Xây Unified Document Model.

2. Phase 2: Tích hợp Docling.
3. Phase 3: Viết Markdown Renderer.
4. Phase 4: Viết Normalizer + Validator.
5. Phase 5: Asset Manager.
6. Phase 6: Output Writer (.md + .json + assets).
7. Phase 7: Test PDF/DOCX/Web.
8. Phase 8: Structure-aware chunking.
9. Phase 9: Embedding + Qdrant.
10. Phase 10: Hybrid retrieval + reranking + RAG.

22. Khuyến nghị kiến trúc cuối cùng



Kết luận: Markdown Output nên được xây thành một module độc lập, deterministic và có khả năng traceability. Đừng coi document.md là nguồn dữ liệu duy nhất; document.json + assets + metadata là nền tảng để chunking và RAG về sau.

23. Bổ sung: PDF có hình ảnh / tài liệu scan sử dụng PaddleOCR

Mục tiêu. Khi PDF chứa trang scan, ảnh chụp, hình ảnh có văn bản hoặc PDF không có lớp text đáng tin cậy, hệ thống phải phát hiện trường hợp này và chuyển các trang cần thiết qua PaddleOCR trước khi tạo Markdown.

23.1. Luồng xử lý PDF

11. Nhận PDF từ thư mục input hoặc API upload.
12. Phân tích từng trang bằng parser chính (ưu tiên Docling) để lấy text, bảng, hình ảnh và metadata.
13. Kiểm tra khả năng trích xuất text của từng trang: số ký tự, mật độ text, tỷ lệ text hợp lệ và khả năng nhận diện layout.
14. Nếu trang là scan/image-only hoặc text extraction không đạt ngưỡng cấu hình, render trang PDF thành ảnh độ phân giải phù hợp.
15. Đưa ảnh trang vào PaddleOCR để nhận diện văn bản.
16. Chuẩn hóa kết quả OCR thành Unified Document Model, giữ page number và bounding box khi có thể.
17. Ghép text/OCR với các thành phần khác như heading, table, figure và image.
18. Render Unified Document Model thành Markdown.
19. Ghi Markdown vào thư mục output và ghi metadata/manifest phục vụ RAG.

23.2. Quy tắc lựa chọn OCR

| Trường hợp | Xử lý | OCR |
|-------------------------------------|-----------------------------------|------------------------------|
| PDF có text layer tốt | Parser trực tiếp | Không cần |
| PDF có cả text và image | Parser + xử lý image/figure | Chỉ OCR vùng/trang cần thiết |
| PDF scan toàn trang | Render page → OCR | PaddleOCR bắt buộc |
| PDF text extraction quá ít hoặc lỗi | Fallback theo page | PaddleOCR |
| Ảnh chứa chữ trong PDF | Image extraction hoặc render page | PaddleOCR cho image cần đọc |

23.3. Tiêu chí phát hiện trang scan

Không nên chỉ kiểm tra toàn bộ PDF. Quyết định OCR theo từng page giúp giảm thời gian xử lý và tránh OCR lại những trang đã có text tốt.

- `text_length < OCR_MIN_TEXT_CHARS`
- `text_density < OCR_MIN_TEXT_DENSITY`
- text extraction thất bại hoặc trả về ký tự bất thường
- trang chủ yếu là hình ảnh và gần như không có text layer
- người dùng bật `force_ocr=true`

Các ngưỡng trên phải cấu hình được trong config/env thay vì hard-code. Nên có thêm cơ chế ghi lại lý do một page được chuyển sang OCR.

23.4. PaddleOCR pipeline

Khuyến nghị tách PaddleOCR thành một adapter độc lập để có thể thay model hoặc engine OCR mà không ảnh hưởng Markdown renderer.

20. PDF page → image rasterization
21. Image preprocessing (rotation/orientation, resize, optional denoise/contrast)
22. PaddleOCR detection
23. PaddleOCR recognition

- 24. Optional table/layout processing
- 25. Sort OCR blocks theo tọa độ đọc
- 26. Convert OCR blocks → Unified Document Model
- 27. Attach source metadata: document_id, page, bbox, confidence

23.5. Metadata OCR cần giữ

- source_type = pdf
- page_number
- ocr = true/false
- ocr_engine = paddleocr
- ocr_model / language
- confidence nếu engine trả về
- bbox [x1, y1, x2, y2] nếu có
- parser = docling hoặc ocr_fallback
- processing_reason = scan | low_text | extraction_error | force_ocr

24. Cấu trúc thư mục output

Markdown output phải được lưu tập trung trong một thư mục output. Tách output khỏi input và temporary files để pipeline dễ vận hành, backup và đưa sang bước chunking/Qdrant.

Cấu trúc đề xuất:

```
project/
├── input/
│   ├── pdf/
│   ├── docx/
│   └── web/
├── output/
│   ├── markdown/
│   │   ├── document_001.md
│   │   ├── document_002.md
│   │   └── ...
│   ├── assets/
│   │   ├── document_001/
│   │   └── document_002/
│   ├── manifest/
│   │   ├── document_001.json
│   │   └── document_002.json
├── tmp/
│   ├── rendered_pages/
│   └── ocr/
```

24.1. Quy tắc ghi Markdown

- Mặc định: output/markdown/<document_id>.md
- Không ghi đè file nếu hệ thống chạy lại trừ khi overwrite=true.
- Tên file phải sanitize, không chứa ký tự path traversal.

- Có thể dùng document_id ổn định để tránh trùng tên nguồn.
- Nếu cần multi-tenant, dùng output/markdown/<tenant>/<document_id>.md.
- Assets tham chiếu tương đối từ Markdown, ví dụ ../assets/<document_id>/image-001.png.

24.2. Front matter đề xuất

```
---
document_id: "doc_001"
source_type: "pdf"
source_name: "example.pdf"
parser: "docling"
ocr_used: true
ocr_engine: "paddleocr"
language: "vi"
page_count: 20
created_at: "2026-09-16T00:00:00Z"
---
```

25. Kiến trúc module cập nhật

- app/ingestion/ — nhận PDF/DOCX/Web
- app/parsers/docling_adapter.py — parser chính
- app/parsers/web_adapter.py — Trafilatura/Playwright
- app/ocr/paddleocr_adapter.py — OCR cho page/image
- app/ocr/detector.py — quyết định page nào cần OCR
- app/models/document.py — Unified Document Model
- app/renderers/markdown_renderer.py — render Markdown
- app/processors/normalizer.py — chuẩn hóa nội dung
- app/output/writer.py — ghi file Markdown
- app/output/asset_writer.py — ghi image/assets
- app/output/manifest_writer.py — ghi JSON metadata
- app/pipeline/pdf_pipeline.py — orchestration PDF → Markdown
- tests/ — unit/integration tests

26. Pseudocode pipeline PDF → Markdown

```
def process_pdf(pdf_path):
    document = docling_parse(pdf_path)

    for page in document.pages:
        if needs_ocr(page):
            image = render_pdf_page(pdf_path, page.number)
            ocr_result = paddleocr(image)
            page.replace_or_merge_text(ocr_result)
            page.metadata["ocr"] = True
            page.metadata["ocr_engine"] = "paddleocr"

    normalized = normalize(document)
```

```
markdown = render_markdown(normalized)

output_path = output_dir / "markdown" / f"{document.id}.md"
write_text(output_path, markdown)

write_manifest(document)
write_assets(document)

return output_path
```

27. Cấu hình đề xuất

```
OUTPUT_DIR=./output
MARKDOWN_OUTPUT_DIR=./output/markdown
ASSET_OUTPUT_DIR=./output/assets
MANIFEST_OUTPUT_DIR=./output/manifest
```

```
OCR_ENABLED=true
OCR_ENGINE=paddleocr
OCR_MIN_TEXT_CHARS=30
OCR_MIN_TEXT_DENSITY=0.0005
OCR_LANG=vi
OCR_FORCE=false
```

```
PDF_RENDER_DPI=200
OUTPUT_OVERWRITE=false
```

28. Yêu cầu kiểm thử bổ sung

- PDF text-native: không gọi PaddleOCR nếu text extraction đạt ngưỡng.
- PDF scan: PaddleOCR được gọi cho các page scan và Markdown chứa text OCR.
- PDF mixed: page có text dùng parser; page scan dùng OCR.
- PDF có image chứa chữ: image/page được OCR khi detector xác định cần thiết.
- OCR lỗi: pipeline ghi lỗi theo page, không làm mất toàn bộ document nếu có thể tiếp tục.
- Markdown luôn được ghi vào output/markdown.
- Manifest ghi rõ page nào dùng OCR.
- Chạy lại document không tạo file ngoài output/tmp ngoài các thư mục đã quy định.
- Đường dẫn asset trong Markdown hợp lệ và không trở ra ngoài output.

29. Acceptance Criteria cập nhật

- AC-01: PDF text-native được chuyển sang Markdown mà không OCR không cần thiết.
- AC-02: PDF scan được phát hiện tự động theo page.
- AC-03: PaddleOCR được sử dụng cho page cần OCR.
- AC-04: Kết quả OCR được đưa vào Unified Document Model trước khi render.

- AC-05: Markdown được lưu tại output/markdown/.
- AC-06: Image/assets được lưu tại output/assets/.
- AC-07: Metadata/manifest được lưu tại output/manifest/.
- AC-08: Có thể truy nguyên chunk RAG về document_id và page_number.
- AC-09: Các ngưỡng OCR và đường dẫn output cấu hình được qua environment/config.
- AC-10: Pipeline có log cho parser, OCR, renderer và output writer.

30. Kiến trúc cuối cùng sau khi bổ sung OCR

PDF / DOCX / Web → Ingestion → Parser → OCR Detector → PaddleOCR (chỉ khi cần) → Unified Document Model → Normalizer → Markdown Renderer → output/markdown/*.md + output/assets/* + output/manifest/*.json → Structure-aware Chunking → Embedding → Qdrant → Retrieval → Reranker → RAG.